



10. Versionamento de código com Git e GitHub

☒ Dúvidas?	☐
☒ Exercícios a entregar?	☒
 Módulo	1 - Primeiros passos com Python e Versionamento de código
 Tipo de aula	Teste Vídeo
 Fora do prazo	 Fora do prazo
 Links e mídias	Pasta "Aula 10", vários links na "Parte 4"
 Última atualização	@18 de novembro de 2025 19:45
 Status	Done
 Data início	@14/11/2025

 Data conclusão	@17/11/2025 3:34
 Esforço	Médio

Anotações

Parte 1 - Visão geral do curso e ferramentas

[Vídeo 1 - Introdução ao curso](#)

[Vídeo 2 - O que é versionamento de código?](#)

[Vídeo 3 - O que é git?](#)

[Vídeo 4 - O que é GitHub?](#)

Parte 2 - Instalação, configuração e autenticação

[Vídeo 3 - Configurando o git](#)

[Vídeo 4 - Autenticação via token](#)

[Vídeo 5 - Autenticação via chave SSH](#)

Parte 3 - Primeiros passos com git e GitHub

[Vídeo 1 - Criar e clonar repositórios](#)

[Vídeo 2 - Salvar alterações no repositório local](#)

[Vídeo 3 - Desfazer alterações no repositório local](#)

[Vídeo 4 - Enviar e baixar alterações no repositório remoto](#)

[Vídeo 5 - Trabalhando com branches: criar, mesclar, excluir e tratar conflitos](#)

[Conflitos](#)

[Vídeo 6 - Trabalhando com branches: comandos úteis no dia a dia](#)

`git fetch`

[Clonar uma branch específica](#)

`git stash`

Parte 4 - Dicas e materiais de apoio

[Exercícios](#)

[Arquivos](#)

Anotações

Parte 1 - Visão geral do curso e ferramentas

Vídeo 1 - Introdução ao curso

Ela se apresentou e falou sobre o objetivo geral do curso: os próximos vídeos.

Vídeo 2 - O que é versionamento de código?

Sistemas de controle de versão **controlam** as **versões** de um **arquivo** ao longo do tempo:

- Registrando o histórico de atualizações de um arquivo
- Administrando quais foram as alterações, a data, o autor, etc.
- Gerando organização, controle e segurança

Há dois tipos de sistemas de controle de versão (conhecidos como **VCS**, **version control systems**):

▼ O centralizado (CVCS), como o *CVS* e o *Subversion*

Neste caso, as versões dos arquivos são armazenadas em um servidor único. Se o servidor ficar fora do ar, os usuários podem não conseguir alterar ou recuperar as versões desejadas. Além disso, se houver perda de dados, perde-se todo o projeto.

▼ O distribuído (DVCS), como o *git* e o *Mercurial*

Surgiu como alternativa aos sistemas centralizados. Ele **duplica** (ou **clona**) os arquivos, de forma que todos os envolvidos no projeto tenham as mesmas versões. Neste caso, cada *clone* é um backup, o fluxo de trabalho fica mais flexível, e pode-se trabalhar sem conexão à rede/Internet.

Vídeo 3 - O que é git?

O git é um sistema de versionamento distribuído gratuito e open source, trabalha com ramificações (as *branches*) e fusões (*merging*) de forma eficiente, e é leve e rápido.

Acessar <https://git-scm.com/> para saber mais.

Basicamente, o fluxo é o seguinte:

1. `git clone` clona um repositório git existente para um novo diretório (pasta) local.
2. `git commit` grava alterações no repositório.
3. Antes de enviar suas alterações para o repositório remoto, é preciso verificar se não há novas alterações que foram enviadas ao servidor com `git pull`, que puxa as alterações e as mescla com as suas.
4. `git push`, então, empurra as suas alterações do diretório local para o repositório remoto.

Vídeo 4 - O que é GitHub?

É uma plataforma de hospedagem de códigos para controle de versionamento com git e colaboração.

O GitHub tem uma comunidade ativa e é usado mundialmente.

git ≠ github! O git controla as versões, o github armazena o código enviado pelo git através do IDE.

Parte 2 - Instalação, configuração e autenticação

Pulei alguns vídeos que vou marcar como feitos:

- Vídeo 1 - Instalação do git no Windows
- Vídeo 2 - Instalação do git no Linux (Ubuntu)

Vídeo 3 - Configurando o git

`git config --help` ou `git help --config` nos permite ver quais opções de configurações estão disponíveis.

Para definir configurações para TODOS os projetos futuros, devemos usar a flag `--global`. Para definir apenas para o projeto selecionado atualmente, usar `--local`. Também temos a flag `--system` para configurações do sistema.

```
git config --global user.name "nome_de_usuario" ⇒ define o nome de usuário exibido no git
git config --global user.email "email_usado_no_github" ⇒ define o email usado no git
git config --global init.defaultBranch "nome_da_ramificação_desejado" ⇒ permite alterar o nome da branch padrão
(de master para main, p.ex.) para todos os projetos
```

Lembrando que se o terminal não exibe nada, é porque funcionou. Qualquer alteração ao nome de usuário e ao *e-mail* só se aplica a novos commits, não aos anteriores à troca.

Mais informações sobre as configurações: <https://git-scm.com/docs/git-config>.

Vídeo 4 - Autenticação via token

É importante lembrar que o git não depende do github para fazer o versionamento do código.

Meu `credential.helper` está armazenado em `file:/home/maopequena/.gitconfig`.

Vídeo 5 - Autenticação via chave SSH

Para gerar uma nova chave SSH: `ssh-keygen -t ed25519 -C "maopequena@gmail.com"` (ou o usuário da outra conta: `lizandra.c.s@gmail.com`).

Para iniciar o agente SSH: `eval "$(ssh-agent -s)"`. Isso retorna o PID do agente em execução.

Agora é preciso adicionar a chave privada ao ssh-agent: `ssh-add ~/.ssh/id_ed25519`. Copiar e colar o valor do arquivo obtido com `cat`.

Parte 3 - Primeiros passos com git e GitHub

Vídeo 1 - Criar e clonar repositórios

Há duas maneiras de ter um repositório git na máquina local:

- Transformar um diretório local, que não está sob controle de versionamento, em um repositório git
- Clonar um repositório git existente

A partir de aproximadamente 1min30s, ela começa com exercícios práticos:

```
mkdir repo-local
cd repo-local
git init
cd .git
ls
cat config

git clone https://endereço_repo_github_desejado
OU
git clone https://endereço_repo_github_desejado "nome_pasta_local_clone"
```

```
# que já joga em uma pasta com esse nome dentro da pasta em que esta  
mos trabalhando  
# No meu caso, vou usar o repo portfolio-projetos, que é privado  
git clone https://github.com/maopequena/portfolio-projetos.git "dio-git-gith  
ub"  
  
git remote -v # exhibe a origem do repo
```

Vídeo 2 - Salvar alterações no repositório local

Ela vai direto para exercícios práticos:

```
mkdir dio-resumos-git-github  
cd dio-resumos-git-github  
git init  
git status  
# Após criar o README:  
git add README.md  
  
# Após o comando acima, a resposta é:  
git status  
On branch master  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   README.md  
# O que indica que as alterações ainda estão na área de preparação (staging)  
# Depois:  
git commit -m"commit inicial"  
  
git log
```



É importante notar que o git não reconhece **diretórios vazios**. Para que apareçam alterações a commitar na CLI, é preciso criar arquivos em novos diretórios.

```
# Vamos adicionar uma pasta e seus arquivos ao .gitignore:
```

```
mkdir resumos
```

```
touch resumos/resumos-aula10.md
```

```
git status
```

```
echo resumos/ > .gitignore
```

```
git status
```

```
# Resultado:
```

```
On branch master
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
.gitignore
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

```
# Se removermos a pasta "resumos" do .gitignore com "echo > .gitignore",  
aparecem duas alterações no staging:
```

```
# a pasta "resumos" e o arquivo ".gitignore"
```

```
# Também é possível usar um arquivo ".gitkeep" para que o git reconheça u  
m diretório vazio
```

```
mkdir aulas
```

```
touch aulas/.gitkeep
```

```
git status
```

```
# Para incluir todos os arquivos do staging no commit, usar:
```

```
git add .
```

```
git commit -m ".gitignore e .gitkeep"
```

```
git status
```

```
git log
```

```
# Vamos explorar o caso de um arquivo alterado:
```

```
# Primeiro, alteramos o arquivo README.md. Adicionei o seguinte:
```

```
# ## Explorar o caso de um arquivo alterado:
```

```
# - Vejamos como o git se comporta no bash
git status # mostra que há alterações a commitar
# Se apagarmos a última alteração, mostra que não há nada a commitar.
```

Vídeo 3 - Desfazer alterações no repositório local

No dia a dia, pode acontecer de iniciarmos um repositório git na pasta errada ou adicionar uma mensagem ou arquivo indesejado a um commit. Para não enviar essas alterações erradas ao repositório remoto, vejamos como desfazer essas ações.

Primeiro, vejamos como desfazer o `git init` em uma pasta errada:

```
mkdir pasta_errada
git init
git status # mostra alterações
```

Isso gera um arquivo `.git` na pasta usada, então é só removê-lo:

```
rm -rf .git # remove recursivamente e à força
git status # não mostra alterações
```

Agora, vejamos como restaurar um arquivo na área de trabalho:

```
git status
```

Se apagarmos tudo do arquivo README.md, o status agora indica que o arquivo foi modificado. Então:

```
git restore README.md # restaura o arquivo à versão anterior
git status # agora não indica alterações, e o arquivo voltou ao que era
```

Agora vejamos como alterar a mensagem do último commit:

```
git log
git commit --amend -m "alteração da mensagem do commit mais recente"
git log # mostra essa alteração
```

```
# Também é possível usar o comando acima sem a flag "-m", o que abre o
```


editor, no meu caso: o nano.

Daí, é só editar a mensagem ("alterar a mensagem do commit mais recente") e salvar.

git log # mostra essa alteração

Vejamos como desfazer o commit mais recente. Há três opções para o comando usado:

```
git reset --soft
```

```
git reset --mixed
```

```
git reset --hard
```

Começando com `--soft`. Precisamos copiar o hash do commit a ser desfeito:

```
git log
```

```
# copiei d2ac15164cef142325dfef5036d76dba486eed21, o primeiro commit
```

```
git reset --soft d2ac15164cef142325dfef5036d76dba486eed21
```

```
git status # mostra todas as alterações que já havia commitado
```

```
git add .
```

```
git commit -m "soft"
```

```
git log
```

O que o `--soft` faz é excluir todos os commits posteriores ao mencionado com o hash. Agora vejamos o `--mixed`, que é o comportamento padrão do `git reset`:

```
git reset --mixed d2ac15164cef142325dfef5036d76dba486eed21
```

```
OU
```

```
git reset d2ac15164cef142325dfef5036d76dba486eed21
```

```
git status
```

```
# Resultado:
```

```
On branch master
```

```
Changes not staged for commit:
```

```
(use "git add <file>..." to update what will be committed)
```

```
(use "git restore <file>..." to discard changes in working directory)
```

```
modified: README.md
```

```
Untracked files:
```

(use "git add <file>..." to include in what will be committed)

.gitignore

aulas/

no changes added to commit (use "git add" and/or "git commit -a")

git add .

git commit -m "mixed"

git log

O que o `--mixed` faz é excluir todos os commits posteriores ao mencionado com o hash, inclusive removendo arquivos que já eram conhecidos, diferente do `--soft`, que ainda reconhece os arquivos adicionados anteriormente. Agora vejamos o `--hard`:

```
git reset --hard d2ac15164cef142325dfef5036d76dba486eed21
```

Resultado:

HEAD is now at d2ac151 commit inicial



Isso **APAGA TODOS** os arquivos criados desde o commit mencionado!

git log # mostra apenas o commit inicial

git reflog

Resultado:

d2ac151 (HEAD → master) HEAD@{0}: reset: moving to d2ac15164cef142325dfef5036d76dba486eed21

e6a648f HEAD@{1}: commit: mixed

d2ac151 (HEAD → master) HEAD@{2}: reset: moving to d2ac15164cef142325dfef5036d76dba486eed21

1108417 HEAD@{3}: commit: soft

d2ac151 (HEAD → master) HEAD@{4}: reset: moving to d2ac15164cef142325dfef5036d76dba486eed21

5e1b82e HEAD@{5}: reset: moving to 5e1b82e16a01337f9cda75cae4617b342ed0dc03

5e1b82e HEAD@{6}: commit (amend): alterar a mensagem do commit ma

is recente

1d050a4 HEAD@{7}: commit (amend): alteração da mensagem do commit mais recente

4d5317b HEAD@{8}: commit: update README.md para testar a alteração de arquivos

6530822 HEAD@{9}: commit: .gitignore e .gitkeep

d2ac151 (HEAD → master) HEAD@{10}: commit (initial): commit inicial

Com o comando `reset`, também podemos remover arquivos da área de preparação (staging):

```
git status # ainda mostra a pasta "resumos" com o arquivo "resumo-aula10.md" a adicionar no staging
```

```
git add .
```

```
git status # mostra a pasta "resumos" com o arquivo "resumo-aula10.md" já no staging
```

```
git reset resumos/resumo-aula10.md
```

```
git status # mostra a pasta "resumos" com o arquivo "resumo-aula10.md" a adicionar no staging
```

Também podemos usar:

```
git restore --staged resumos/nome-do-arquivo.ext
```

Vídeo 4 - Enviar e baixar alterações no repositório remoto

Agora vejamos como enviar os arquivos para o repositório remoto. Após o terminal exibir `On branch master, nothing to commit, working tree clean` ao usar o comando `git status`, faremos o seguinte:

```
git remote add origin https://url-para-repo-github/
```

```
# no meu caso, usarei um repo que já existia:
```

```
# https://github.com/maopequena/repo-remoto-dio.git
```

```
# depois de adicionar o repo remoto acima, podemos definir a branch
```

```
# e usar o push:
```

```
git branch -M main # para definir a branch
```

```
git push -u origin master # ou main, ou qq outro nome de branch,  
# para fazer o push
```



DICA IMPORTANTE: no Github, é possível editar um arquivo no VS Code integrado pressionando a tecla `.` (ponto final)!

Agora vamos puxar as alterações feitas no editor online do Github para o repositório local usando o bash:

```
git pull # isso puxa e mescla as alterações
```

Vídeo 5 - Trabalhando com branches: criar, mesclar, excluir e tratar conflitos

Uma branch (ramificação) é um ponteiro móvel para um commit no histórico do repositório. Ao criar uma nova branch (2) a partir de uma existente (1), a nova (2) se inicia apontando para o commit em que a antiga (1) estava.

Uma branch permite testar um novo recurso de um projeto de forma independente, sem alterar o projeto original.

Após criar três novos commits na pasta `/home/maopequena/Documents/dio-resumos-git-github/resumos`, agora vamos trabalhar com as branches:

```
git checkout -b nome-da-branch  
# no meu caso, usarei "branch-2"
```

Agora no terminal já aparece "branch-2" como a ramificação em uso em vez de "main":



E o comando `git log` agora mostra o repo apontando para a branch atual e para a main:

```
git log
commit 47601f [redacted] f1b0 (HEAD -> branch-2, main)
Author: maopequena [redacted]
Date: Mon Nov 17 01:13:40 2025 -0300

commit 2, branch main
```

Depois de um terceiro commit, agora na branch-2, vemos que agora o repo só aponta para a branch 2, enquanto o commit anterior aponta para a branch main:

```
git log
commit 576e [redacted] 8080 (HEAD -> branch-2)
Author: maopequena [redacted]
Date: Mon Nov 17 01:27:42 2025 -0300

commit 3, branch 2

commit 4760 [redacted] f1b0 (main)
Author: maopequena [redacted]
Date: Mon Nov 17 01:13:40 2025 -0300

commit 2, branch main
```

Agora vamos voltar à branch main:

```
git checkout main
# Resultado:
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 3 commits.
(use "git push" to publish your local commits)
```



Note que ao mudar de branch, os arquivos são alterados na pasta local.

Com o comando `git branch -v`, são exibidos os últimos commits em cada branch:

```
git branch -v
branch-2 576e562 commit 3, branch 2
* main 47601f1 [ahead 3] commit 2, branch main
```

Se quisermos que as alterações da branch 2 apareçam na main, é preciso mesclá-las:

```
git merge nome-da-branch # a ser mesclada com a main
# no meu caso, branch-2
```

O resultado de `git status` agora é:

```
~/Documentos/dio-resumos-git-github/resumos git:(main) (0.011s)
git status
On branch main
Your branch is ahead of 'origin/main' by 4 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

E de `git log`:

```
~/Documentos/dio-resumos-git-github/resumos git:(main) (0.013s)
git log
commit 576e [redacted] 8080 (HEAD -> main, branch-2)
Author: maopequena [redacted]
Date: Mon Nov 17 01:27:42 2025 -0300

    commit 3, branch 2

commit 4760 [redacted] f1b0
Author: maopequena [redacted]
Date: Mon Nov 17 01:13:40 2025 -0300

    commit 2, branch main

commit 92d3 [redacted] ab50
Author: maopequena [redacted]
Date: Mon Nov 17 01:12:35 2025 -0300

    commit 1, branch main

commit 502b [redacted] db09
Author: maopequena [redacted]
Date: Mon Nov 17 01:10:41 2025 -0300

    commit 0, vídeo 5 - branches

commit 0503 [redacted] 1d34 (origin/main)
Author: [redacted]
Date: Sun Nov 16 20:01:03 2025 -0300

    uso do editor online do github com .
```

Note que os arquivos todos foram mesclados na pasta local. Agora vamos excluir a branch-2, que não será mais utilizada. Porém, antes, vamos listar as branches que temos no repo:

```
git branch
# Resultado:
branch-2
* main
```

Note o asterisco (*), que indica que esta é a branch selecionada no momento. E observe que as branches são listadas alfabeticamente.

```
git branch -d branch-2 # -d de "delete"
# Resultado:
Deleted branch branch-2 (was 576e562).
```

O comando `git branch` agora só lista a `main`.



Note que essa `branch-2` só existiu localmente, já que não demos o comando `git push` para enviar isso ao repo remoto.

Conflitos

Agora vejamos como lidar com conflitos (conhecidos como **conflitos de mesclagem** ou **de merge**) que podem aparecer ao trabalhar com branches.

Alteramos o README.md no Github e criamos um commit. Depois fizemos a clonagem do repo para a máquina local. Após a clonagem, alteramos novamente o arquivo, tanto localmente, quanto remotamente. E agora vamos, da máquina local, tentar enviar os arquivos alterados, em cima das alterações feitas diretamente no repo remoto:

```
git add .
git commit -m "commit da máquina local após edição de outro colaborador remotamente"
```

O resultado é o seguinte:

```
~/Documentos/dio-resumos-git-github git:(main) 1 file changed, 3 insertions(+) (0.02s)
git commit -m "commit da máquina local após edição de outro colaborador remotamente"
[main 5125a55] commit da máquina local após edição de outro colaborador remotamente
1 file changed, 3 insertions(+)
```

E o `git log` indica que o repositório local está no commit após a edição remota, enquanto o remoto indica que ainda está no commit antes de clonar:

```
~/Documentos/dio-resumos-git-github git:(main)
git log
commit 5125a55d10 (HEAD -> main)
Author: maopequena
Date: Mon Nov 17 02:21:20 2025 -0300

    commit da máquina local após edição de outro colaborador remotamente

commit 746d1b086
Merge: 576e562 8b2b37c
Author: maopequena
Date: Mon Nov 17 02:08:48 2025 -0300

    Merge branch 'main' of https://github.com/maopequena/repo-remoto-dio
    merge antes da clonagem no vídeo de conflitos

commit 8b2b37c (origin/main)
Author:
Date: Mon Nov 17 02:03:01 2025 -0300

    commit antes de clonar
```

Ao tentar o `git push`, ocorre uma rejeição do push:

```
~/Documentos/dio-resumos-git-github git:(main) (0.448s)
git push origin main
To https://github.com/maopequena/repo-remoto-dio.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/maopequena/repo-remoto-dio.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Então, primeiro, precisamos fazer `git pull` e, depois, um `git rebase` para juntar todas as alterações. Eu usei `git rebase --continue`, em vez de `git commit --amend`, porque as alterações locais eram suficientes.

Vídeo 6 - Trabalhando com branches: comandos úteis no dia a dia

git fetch

O comando `git pull` nada mais é do que a união dos comandos `git fetch` e `git merge`. O primeiro busca as alterações e o segundo as mescla.

Usando este comando, ele busca as alterações, sem mesclá-las no diretório local.

Em seguida, podemos usar `git diff main origin/main` para ver quais alterações foram realizadas. Note que, neste caso, queremos ver quais alterações foram feitas no repo remoto (`origin`) e que não estão na máquina local (`main`), mas poderia ser o inverso com `git diff main main/origin`.

Após analisar as alterações, para mesclar as informações, use `git merge origin/main`.

Clonar uma branch específica

```
git clone https://url-repo-github --branch nome-da-branch-desejada --single-branch
```



Note que, se o nome da branch não for informado, será clonada apenas a branch principal.

git stash

Este comando serve para quando se precisa fazer um commit e não se deseja enviar uma determinada alteração. Ele, sem nenhuma outra adição, armazena a alteração, mas retorna todos os arquivos ao estado em que estavam antes do "stash".

O comando `git stash list` mostra alterações que foram armazenadas para edição/envio posteriores.

`git stash pop` exclui a alteração armazenada.

`git stash apply` aplica a alteração armazenada à branch atual.

Parte 4 - Dicas e materiais de apoio

Ler:

- <https://git-scm.com/docs>, seção *Branching and Merging*
- <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>
- </home/maopequena/Documentos/Programação/git-cheat-sheet.pdf>
- <https://github.com/elidianaandrade/git-github-learning-quest>
- <https://aline-antunes.gitbook.io/formacao-fundamentos-github>, material textual sobre GitHub
- <https://docs.github.com/pt/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>
- <https://github.blog/>
- <https://github.com/conventional-commits> / <https://github.com/conventional-commits/conventionalcommits.org>
-
- Pesquisar sobre convenções na hora de criar commits e nomear branches (último link acima)
- <https://gitfluence.com/>, AI para ajudar a descobrir qual comando usar
- <https://docs.github.com/pt/pages/getting-started-with-github-pages>

Exercícios

100% ⇒ 10/10 corretas na primeira tentativa

Arquivos

Pasta "Aula 10"